main.cpp

Run ▶

```cpp

#include <iostream>
#include <string>
#include <map>
#include <iomanip>
#include <stdexcept>
using namespace std;

// Constants defined in one place
constexpr long long PAISE_PER_RUPEE = 100;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int STUDENT_CONCESSION_PERCENT = 50;
constexpr int SENIOR_CONCESSION_PERCENT = 50;

constexpr int MORNING_PEAK_START = 8 * 60;
constexpr int MORNING_PEAK_END = 10 * 60;
constexpr int EVENING_PEAK_START = 17 * 60;
constexpr int EVENING_PEAK_END = 20 * 60;

constexpr long long MINIMUM_BALANCE_PAISE = 50 * PAISE_PER_RUPEE;

// Zone enumeration
enum class Zone {
    Inner,
    Middle,
    Outer
```

Output

```
NAMMA METRO SMART-CARD FARE SYSTEM
Card ID: CARD101
Passenger Type: Student
Initial Balance: Rs. 100.00
Peak Surcharge: 20%
Student Concession: 50%
Calculated Fare: Rs. 12.00
Journey successful.
Remaining Balance: Rs. 88.00
```

main.cpp

Run ▶

```
        cout << "int: " << sizeof(int) << " byte(s)\n";
        cout << "long long: " << sizeof(long long) << " byte(s)\n";
        cout << "float: " << sizeof(float) << " byte(s)\n";
        cout << "double: " << sizeof(double) << " byte(s)\n";

        cout << "long long maximum: "
             << numeric_limits<long long>::max() << '\n';

        // Compare double and integer money representation
        double doubleFare = 0.1 + 0.2;
        cout << setprecision(17);
        cout << "Double precision example (0.1 + 0.2): "
             << doubleFare << '\n';

        cout << setprecision(2);
        cout << "Integer paise example (10 + 20 paise): "
             << (10 + 20) / 100.0 << '\n';
    }
    catch (const exception& e) {
        cerr << "Error: " << e.what() << '\n';
        return 1;
    }

    return 0;
}
```

Output

```
NAMMA METRO DATA TYPE VALIDATION
Card ID: CARD101
Passenger ID: STU101
Journey ID: 1001
Initial Balance: Rs. 100.00
Calculated Fare: Rs. 12.00
Journey successful.
Remaining Balance: Rs. 88.00
Collection after one journey: Rs. 12.00
Zone enum converted explicitly: 1
Minimum balance rule satisfied: true

DATA TYPE SIZES
bool: 1 byte(s)
char: 1 byte(s)
int: 4 byte(s)
long long: 8 byte(s)
float: 4 byte(s)
double: 8 byte(s)
long long maximum: 9223372036854775807
Double precision example (0.1 + 0.2): 0.30000000000000004
Integer paise example (10 + 20 paise): 0.30
```

main.cpp

Run ▶

```
    // Integer to bool conversion
    bool flag = 5;
    cout << flag << " " << boolalpha << flag << '\n';

    // Signed and unsigned comparison
    unsigned int u = 1;
    int s = -1;
    cout << (s < u ? "s < u" : "s >= u") << '\n';

    // Explicit conversion examples
    double value = 9.8;
    int converted = static_cast<int>(value);
    cout << "Explicit conversion: " << converted << '\n';

    // Brace initialization prevents narrowing
    int validNumber{3};
    cout << "Brace initialization: " << validNumber << '\n';

    // Demonstrate overflow-safe range information
    cout << "Maximum int: "
         << numeric_limits<int>::max() << '\n';

    return 0;
}
```

Output

```
3 3.5
C 68
1 true
s >= u
Explicit conversion: 9
Brace initialization: 3
Maximum int: 2147483647
```

main.cpp

Run ▶

```
    for (int i = 0; i < 10; i += 3)
        k += i;

    cout << k << '\n';

    // Fare expression examples
    int hour = 8;
    int baseFare = 2000; // Rs. 20 in paise
    int concessionPercent = 50;

    bool peak = isPeakHour(hour);

    int fareAfterSurcharge = applySurcharge(baseFare, peak);
    int finalFare = applyConcession(
        fareAfterSurcharge, concessionPercent
    );

    cout << "Peak hour: " << boolalpha << peak << '\n';
    cout << "Base fare in paise: " << baseFare << '\n';
    cout << "Fare after surcharge in paise: "
         << fareAfterSurcharge << '\n';
    cout << "Final fare in paise: " << finalFare << '\n';

    return 0;
}
```

Output

```
11 16
5 2
3 1 -3 -1
18
Peak hour: true
Base fare in paise: 2000
Fare after surcharge in paise: 2400
Final fare in paise: 1200
```

main.cpp

Run ▶

```
    Trip trips[] = {
        {Zone::Inner, false},
        {Zone::Outer, true},
        {Zone::Middle, false}
    };

    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);

        if (canTravel(balance, f)) {
            balance -= f;

            cout << "Fare Rs "
                 << f / 100 << '.'
                 << setw(2) << setfill('0') << f % 100
                 << setfill(' ')
                 << " balance Rs " << balance / 100 << '\n';
        } else {
            cout << "Denied (low balance)";
            cout << " balance Rs " << balance / 100 << '\n';
        }
    }

    return 0;
}
```

Output

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

main.cpp

Run ▶

```
    }
}

// ========================================
// MAIN FUNCTION
// ========================================

int main() {
    cout << "NAMMA METRO FARE SYSTEM\n\n";

    cout << "PROCEDURAL VERSION\n";
    proceduralJourney(1000);  // Rs 10
    proceduralJourney(3600);  // Rs 36
    proceduralJourney(1000);  // Rs 10

    cout << "\nOBJECT-ORIENTED VERSION\n";

    SmartCard card(10000);    // Rs 100
```

Output

```
NAMMA METRO FARE SYSTEM

PROCEDURAL VERSION
Procedural: Journey successful. Balance Rs 90
Procedural: Journey successful. Balance Rs 54
Procedural: Journey denied. Balance Rs 54

OBJECT-ORIENTED VERSION
OOP: Journey successful. Balance Rs 90
OOP: Journey successful. Balance Rs 54
OOP: Journey denied. Balance Rs 54
```

main.cpp

Run ▶

```
    }
}

// ========================================
// MAIN FUNCTION
// ========================================

int main() {
    cout << "NAMMA METRO FARE SYSTEM\n\n";

    cout << "PROCEDURAL VERSION\n";
    proceduralJourney(1000);  // Rs 10
    proceduralJourney(3600);  // Rs 36
    proceduralJourney(1000);  // Rs 10

    cout << "\nOBJECT-ORIENTED VERSION\n";

    SmartCard card(10000);    // Rs 100

    objectOrientedJourney(card, 1000);
    objectOrientedJourney(card, 3600);
    objectOrientedJourney(card, 1000);

    return 0;
}
```

Output

```
NAMMA METRO FARE SYSTEM

PROCEDURAL VERSION
Procedural: Journey successful. Balance Rs 90
Procedural: Journey successful. Balance Rs 54
Procedural: Journey denied. Balance Rs 54

OBJECT-ORIENTED VERSION
OOP: Journey successful. Balance Rs 90
OOP: Journey successful. Balance Rs 54
OOP: Journey denied. Balance Rs 54
```

main.cpp

Run ▶

```
                << "of Rs. 50 must remain.\n";
            continue;
        }

        balance -= fare;
        journeyCount++;

        cout << "Journey accepted. Fare: ";
        printMoney(fare);
        cout << '\n';

        cout << "Remaining balance: ";
        printMoney(balance);
        cout << "\n";
    }

    cout << "\n========== JOURNEY REPORT ==========\n";
    cout << "Card ID: " << cardId << '\n';
    cout << "Successful journeys: " << journeyCount << '\n';
    cout << "Final balance: ";
    printMoney(balance);
    cout << "\n====================================\n";

    return 0;
}
```

Output

```
NAMMA METRO SMART-CARD SYSTEM
Enter Card ID: 123
Enter initial balance in paise: 20000
Enter fare in paise (0 to finish): 1000
Journey accepted. Fare: Rs. 10.00
Remaining balance: Rs. 190.00
Enter fare in paise (0 to finish): 3600
Journey accepted. Fare: Rs. 36.00
Remaining balance: Rs. 154.00
Enter fare in paise (0 to finish): 1000
Journey accepted. Fare: Rs. 10.00
Remaining balance: Rs. 144.00
Enter fare in paise (0 to finish): 0

========== JOURNEY REPORT ==========
Card ID: 123
Successful journeys: 3
Final balance: Rs. 144.00
====================================
```

main.cpp

Run ▶

```
    if (!rechargeResult && rechargeBalance == 10000) {
        passed++;
    }
    totalTests++;

    // Invalid hour input test
    cout << "\nINVALID HOUR INPUT TEST\n";
    int hour = 0;
    bool validHour = parseHour("abc", hour);

    cout << (!validHour
             ? "PASS: Letters rejected as hour\n"
             : "FAIL\n");

    if (!validHour) {
        passed++;
    }
    totalTests++;

    cout << "\nTEST SUMMARY\n";
    cout << "Tests passed: " << passed
         << " out of " << totalTests << '\n';

    return 0;
}
```

Output

```
FARE TEST TABLE
Inner off-peak | Expected: Rs. 10.00 | Actual: Rs. 10.00 | PASS
Inner at 8:00 | Expected: Rs. 12.00 | Actual: Rs. 12.00 | PASS
Middle at 10:00 | Expected: Rs. 20.00 | Actual: Rs. 20.00 | PASS
Outer at 17:00 | Expected: Rs. 36.00 | Actual: Rs. 36.00 | PASS
Outer at 20:00 | Expected: Rs. 30.00 | Actual: Rs. 30.00 | PASS

FIVE JOURNEY TOTAL
Total fare: Rs. 108.00

MINIMUM BALANCE TEST
Balance Rs. 60, fare Rs. 10: PASS
Remaining balance: Rs. 50.00

INVALID ZONE TEST
PASS: Unknown zone rejected

NEGATIVE RECHARGE TEST
PASS: Negative recharge rejected

INVALID HOUR INPUT TEST
PASS: Letters rejected as hour

TEST SUMMARY
Tests passed: 8 out of 8
```

main.cpp

Run ▶

```
        }

        cout << "Long long paise total: "
             << totalPaise << '\n';

        cout << "----------------------------------\n";
    }

    // Demonstrate the limits of int
    cout << "\nDATA TYPE LIMITS\n";
    cout << "Maximum int: "
         << numeric_limits<int>::max() << '\n';

    cout << "Maximum long long: "
         << numeric_limits<long long>::max() << '\n';

    // Demonstrate floating-point representation
    double test = 0.1 + 0.2;

    cout << setprecision(17);
    cout << "\nFloating-point test (0.1 + 0.2): "
         << test << '\n';

    return 0;
}
```

Output

```
Double total:         Rs. 246800.00
Difference:           Rs. 0.00
Int paise total: 24680000
Long long paise total: 24680000
----------------------------------
Journeys: 500000
Integer-paise total: Rs. 6170000.00
Double total:         Rs. 6170000.00
Difference:           Rs. 0.00
Int paise total: 617000000
Long long paise total: 617000000
----------------------------------
Journeys: 3000000
Integer-paise total: Rs. 37020000.00
Double total:         Rs. 37020000.00
Difference:           Rs. 0.00
Int paise total: OVERFLOW RISK
Long long paise total: 3702000000
----------------------------------

DATA TYPE LIMITS
Maximum int: 2147483647
Maximum long long: 9223372036854775807

Floating-point test (0.1 + 0.2): 0.30000000000000004
```

main.cpp

Run ▶

```
118
119         if (fare < 0) {
120             cout << "Invalid zone\n";
121             continue;
122         }
123
124         printMoney(fare);
125
126         if (card.deduct(fare)) {
127             cout << " | Accepted | Balance: ";
128             printMoney(card.getBalance());
129             cout << '\n';
130         } else {
131             cout << " | Denied: minimum balance rule | Balance: ";
132             printMoney(card.getBalance());
133             cout << '\n';
134         }
135     }
136
137     cout << "\nFinal balance: ";
138     printMoney(card.getBalance());
139     cout << '\n';
140
141     return 0;
142 }
```

Output

```
NAMMA METRO - FINAL DESIGN

Inner zone weekday | Fare: Rs. 10.00 | Accepted | Balance: Rs. 90.00
Outer zone peak hour | Fare: Rs. 36.00 | Accepted | Balance: Rs. 54.00
Express zone weekend | Fare: Rs. 36.00 | Denied: minimum balance rule |
Balance: Rs. 54.00
Monthly pass journey | Fare: Rs. 0.00 | Accepted | Balance: Rs. 54.00

Final balance: Rs. 54.00
```

main.cpp

Run ▶

```
        if (fare < 0) {
            cout << "Invalid zone\n";
            continue;
        }

        printMoney(fare);

        if (card.deduct(fare)) {
            cout << " | Accepted | Balance: ";
            printMoney(card.getBalance());
            cout << '\n';
        } else {
            cout << " | Denied (minimum Rs. 50 must remain)"
                 << " | Balance: ";
            printMoney(card.getBalance());
            cout << '\n';
        }
    }

    cout << "\nFinal balance: ";
    printMoney(card.getBalance());
    cout << '\n';

    return 0;
}
```

Output

```
NAMMA METRO - REVISED FARE SYSTEM

Inner zone weekday | Fare: Rs. 0.00 | Accepted | Balance: Rs. 100.00
Outer zone peak hour | Fare: Rs. 36.00 | Accepted | Balance: Rs. 64.00
Airport zone weekend | Fare: Rs. 54.00 | Denied (minimum Rs. 50 must
remain) | Balance: Rs. 64.00
Middle zone monthly pass | Fare: Rs. 0.00 | Accepted | Balance: Rs. 64.00

Final balance: Rs. 64.00
```